R Workshop for Applied Research Projects

A practical bridge for Stata users

J. Montornes

2026-05-08

Workshop goals

- Understand the key differences between R and Stata.
- Write reproducible scripts instead of isolated console commands.
- Manipulate data with `dplyr`.
- Produce charts, regressions and simple output tables.
- Know where to find help and how to structure a small project.

Agenda

1. Interface, help and project structure.
2. Packages, objects and syntax.
3. Data import and data manipulation.
4. Charts and tables.
5. Econometric estimation.
6. Dates, strings and wrap-up exercise.

RStudio interface

- **Script**: saved, versionable and rerunnable source code.
- **Console**: interactive execution and error messages.
- **Environment**: objects currently stored in memory.
- **Files / Plots / Packages / Help**: outputs, files and documentation.

Useful habit: anything you want to keep should be written in a script, not only typed in the console.

Help and documentation

```
?lm
help("merge")
example(mean)
```

Useful resources:

- R for Data Science
- Stata2R
- UtilitR

Packages

A package adds functions, datasets or methods.

```
install.packages("dplyr")  # once per machine
library(dplyr)             # in each new session
```

For training, listing required packages in the README or an install script prevents many last-minute issues.

Objects and assignment

```
baguette_price <- 0.95
quantities <- c(1, 2, 3)
spending <- baguette_price * quantities
spending
```

```
## [1] 0.95 1.90 2.85
```

In R, results are usually assigned to objects with `<-`. The `=` sign often works, but `<-` remains the clearest convention.

Useful differences with Stata

Stata	R
One active dataset in memory	Several objects at the same time
use, **save**	`read.csv()`, `write.csv()`, `readxl::read_xlsx()`
gen, **replace**	`mutate()`
keep if	`filter()`
collapse	`group_by()` then `summarise()`
merge	`left_join()`, `inner_join()`, `full_join()`

Importing a file

```r
economy <- read.csv("data/sample_economy.csv")
head(economy)
```

```
##   country year      sector employment wage price_index consumption
## 1  France 2020    Industry       3100 37.2       101.4       220.5
## 2  France 2020    Services       8200 34.8       101.4       516.2
## 3  France 2020 Agriculture        640 25.1       101.4        41.0
## 4  France 2021    Industry       3180 38.1       103.1       229.8
## 5  France 2021    Services       8460 35.7       103.1       538.4
## 6  France 2021 Agriculture        630 25.8       103.1        42.3
```

For Excel files:

```r
readxl::read_xlsx("data/my_file.xlsx", sheet = "Sheet1")
```

Selecting and filtering

Stata

```
keep if country == "France" & year >= 2021
keep country year sector wage
```

R

```
economy |>
  filter(country == "France", year >= 2021) |>
  select(country, year, sector, wage)
```

```
##   country year      sector wage
## 1  France 2021    Industry 38.1
## 2  France 2021    Services 35.7
## 3  France 2021 Agriculture 25.8
## 4  France 2022    Industry 40.0
## 5  France 2022    Services 37.6
## 6  France 2022 Agriculture 27.2
```

Creating a variable

Stata

```
gen real_wage = wage / price_index * 100
```

R

```
economy <- economy |>
  mutate(real_wage = wage / price_index * 100)

economy |>
  select(country, year, sector, real_wage) |>
  head()
```

```
##   country year      sector real_wage
## 1  France 2020    Industry  36.68639
## 2  France 2020    Services  34.31953
## 3  France 2020 Agriculture  24.75345
## 4  France 2021    Industry  36.95441
## 5  France 2021    Services  34.62658
## 6  France 2021 Agriculture  25.02425
```

Aggregating

Stata

```
collapse (mean) mean_wage = wage, by(country year)
```

R

```
country_year <- economy |>
  group_by(country, year) |>
  summarise(mean_wage = mean(wage), .groups = "drop")

country_year
```

```
## # A tibble: 9 × 3
##   country  year mean_wage
##   <chr>   <int>     <dbl>
## 1 France   2020      32.4
## 2 France   2021      33.2
## 3 France   2022      34.9
## 4 Germany  2020      33.4
## 5 Germany  2021      34.1
## 6 Germany  2022      36.0
## 7 Italy    2020      27.8
## 8 Italy    2021      28.5
## 9 Italy    2022      30.0
```

Joining two tables

```r
population <- data.frame(
  country = c("France", "Germany", "Italy"),
  population_m = c(67.8, 83.2, 59.0)
)

country_year |>
  left_join(population, by = "country") |>
  head()
```

```
## # A tibble: 6 × 4
##   country  year mean_wage population_m
##   <chr>   <int>     <dbl>        <dbl>
## 1 France   2020      32.4         67.8
## 2 France   2021      33.2         67.8
## 3 France   2022      34.9         67.8
## 4 Germany  2020      33.4         83.2
## 5 Germany  2021      34.1         83.2
## 6 Germany  2022      36.0         83.2
```

Remember the direction of the join: `left_join(x, y)` keeps all rows from `x`.

data.table in one sentence

`data.table` is very efficient on large datasets. Its syntax is compact:

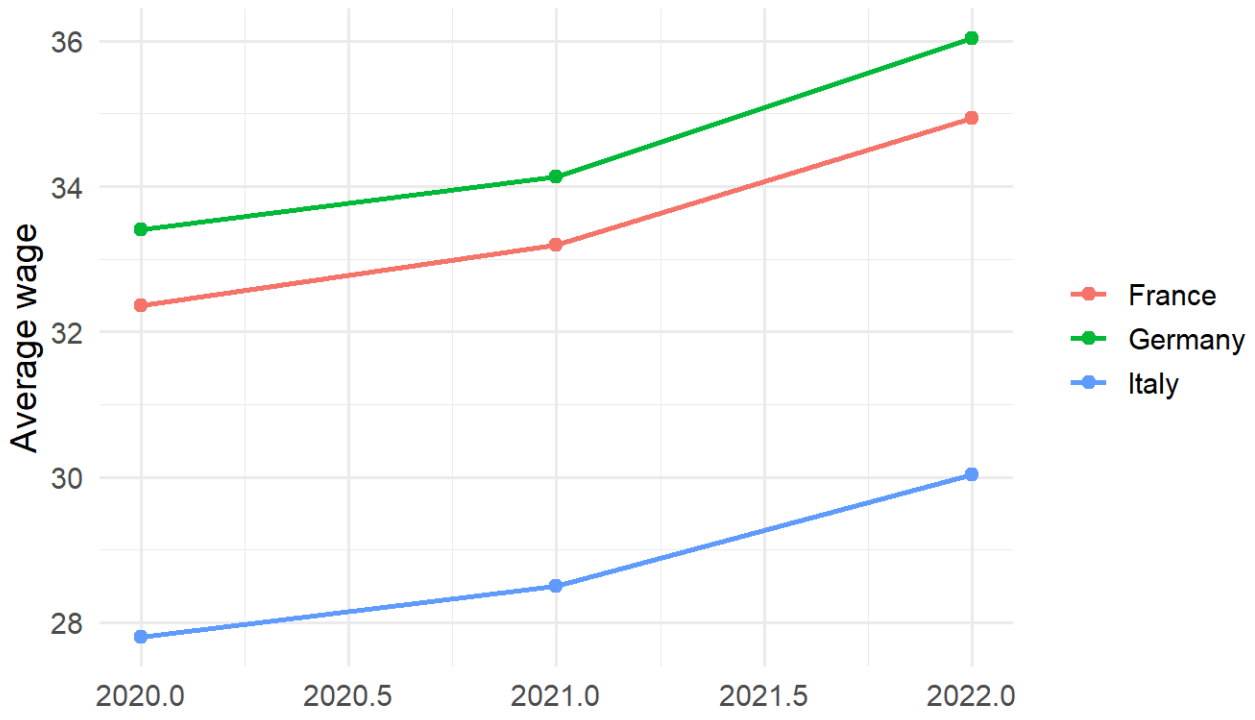
```
library(data.table)
dt <- as.data.table(economy)
dt[country == "France", .(mean_wage = mean(wage)), by = year]
```

```
##     year mean_wage
##    <int>     <num>
## 1:  2020  32.36667
## 2:  2021  33.20000
## 3:  2022  34.93333
```

For beginners, `dplyr` is often easier to read; for massive datasets, `data.table` may become preferable.

Simple chart

```
ggplot(country_year, aes(x = year, y = mean_wage, color = country)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(x = NULL, y = "Average wage", color = NULL)
```



ggplot2 principles

- `ggplot(data, aes(...))` declares the data and mapped variables.
- `geom_*()` chooses the visual representation: points, lines, bars.
- `labs()`, `scale_*()` and `theme_*()` adjust presentation.
- Each layer is added with `+`.

```
ggplot(data, aes(x = x, y = y)) +
  geom_point() +
  labs(x = "X", y = "Y")
```

Linear regression

Stata

```
reg real_wage employment i.sector
```

Base R

```
model_lm <- lm(real_wage ~ employment + factor(sector), data = economy)
summary(model_lm)
```

```
##
## Call:
## lm(formula = real_wage ~ employment + factor(sector), data = economy)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.8674 -1.6908  0.4735  1.4333  2.7922
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)            2.324e+01  7.366e-01  31.553  < 2e-16 ***
## employment             7.674e-04  2.274e-04   3.375  0.00261 **
## factor(sector)Industry 8.481e+00  1.430e+00   5.932 4.78e-06 ***
## factor(sector)Services 2.559e+00  2.291e+00   1.117  0.27562
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2.164 on 23 degrees of freedom
## Multiple R-squared:  0.8762, Adjusted R-squared:   0.86
## F-statistic: 54.25 on 3 and 23 DF,  p-value: 1.372e-10
```

Fixed effects with fixest

`fixest` is close in spirit to `reghdfe`: fast, concise and convenient for fixed effects and clustered standard errors.

```
model_fe <- feols(
  real_wage ~ employment | country + year,
  data = economy,
  vcov = ~ country
)
etable(model_fe)
```

```
##                           model_fe
## Dependent Var.:          real_wage
##
## employment      0.0010** (4.85e-5)
## Fixed-Effects:  ------------------
## country                        Yes
## year                           Yes
## _______________ __________________
## S.E.: Clustered        by: country
## Observations                    27
## R2                         0.61545
## Within R2                  0.54274
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Clean output

```
broom::tidy(model_lm)
```

```
## # A tibble: 4 × 5
##   term                    estimate std.error statistic  p.value
##   <chr>                      <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)            23.2       0.737        31.6  1.94e-20
## 2 employment              0.000767  0.000227      3.37 2.61e- 3
## 3 factor(sector)Industry  8.48      1.43          5.93 4.78e- 6
## 4 factor(sector)Services  2.56      2.29          1.12 2.76e- 1
```

For a report:

- keep model objects (`model_lm`, `model_fe`);
- generate tables in the script;
- avoid manual copy-paste from the console.

Dates with lubridate

```
day <- ymd("2023-02-11")
day
```

```
## [1] "2023-02-11"
```

```
year(day)
```

```
## [1] 2023
```

```
month(day)
```

```
## [1] 2
```

`ymd()`, `dmy()` and `mdy()` make the day/month/year order explicit.

Strings with stringr

```r
text <- "Hello world"
str_replace(text, "world", "universe")
```

```
## [1] "Hello universe"
```

```r
str_sub(text, 1, 5)
```

```
## [1] "Hello"
```

```r
str_detect(text, "Hello")
```

```
## [1] TRUE
```

Mental translation:

- Stata `subinstr()` -> R `str_replace()` or `str_replace_all()`.
- Stata `substr()` -> R `str_sub()`.
- Stata `regexm()` -> R `str_detect()`.

Web calls and external databases

For DB.NOMICS or other APIs, showing non-executed code keeps slides robust.

```
library(rdbnomics)
df <- rdbnomics::rdb(ids = "INSEE/CNT-2014-CSI/...")
```

In training, always provide a local fallback file in case the network, proxy or API is unavailable.

Recommended mini-project

1. Import `data/sample_economy.csv`.
2. Create a real wage variable.
3. Compute average wages by country and year.
4. Plot the time series.
5. Estimate a simple model with fixed effects.
6. Save the chart and the result table.

Project good practices

```
AtelierR/
  data/
  exercises/
  solutions/
  atelierR.Rmd
  atelierR_en.Rmd
  README.md
```

- Use relative paths from the project root.
- Store lightweight example data in `data/`.
- Keep executable solutions in `solutions/`.
- Document goals, prerequisites and rendering commands in the README.

Conclusion

R is not only a statistical calculator: it is a project environment.

The main gain over purely interactive work is reproducibility: tomorrow, someone should be able to rerun the script and obtain the same outputs.